

A1. Анализ строковых сортировок

Ссылка на репозиторий:

<https://github.com/AMGolovina/Algorithm/tree/main/SET9/A1>

ID A1m. Реализация STRING MERGE SORT: [375454075](#)

ID A1q. Реализация STRING QUICK SORT: [375464175](#)

ID A1r. Реализация MSD RADIX SORT: [375568016](#)

ID A1rq. Реализация MSD RADIX+QUICK SORT: [375571245](#)

Этап 1. Подготовка тестовых данных

Тестовые данные находятся в файлах:

- **random.txt** - полностью не упорядоченный набор строк
- **almost_sorted.txt** - почти отсортированный набор строк
- **reversed.txt** - строки в обратном лексикографическом порядке

Реализация класса находится в файле **StringGenerator.cpp**

Этап 2. Эмпирический анализ стандартных алгоритмов сортировки

Был произведен анализ стандартных алгоритмов сортировки **QUICKSORT** и **MERGESORT** для тестовых данных и полученные результаты (время выполнения и количество символьных сравнений) записаны в файл **result.csv**

Этап 3. Эмпирический анализ адаптированных алгоритмов сортировки

Реализация 4х алгоритмов в файлах:

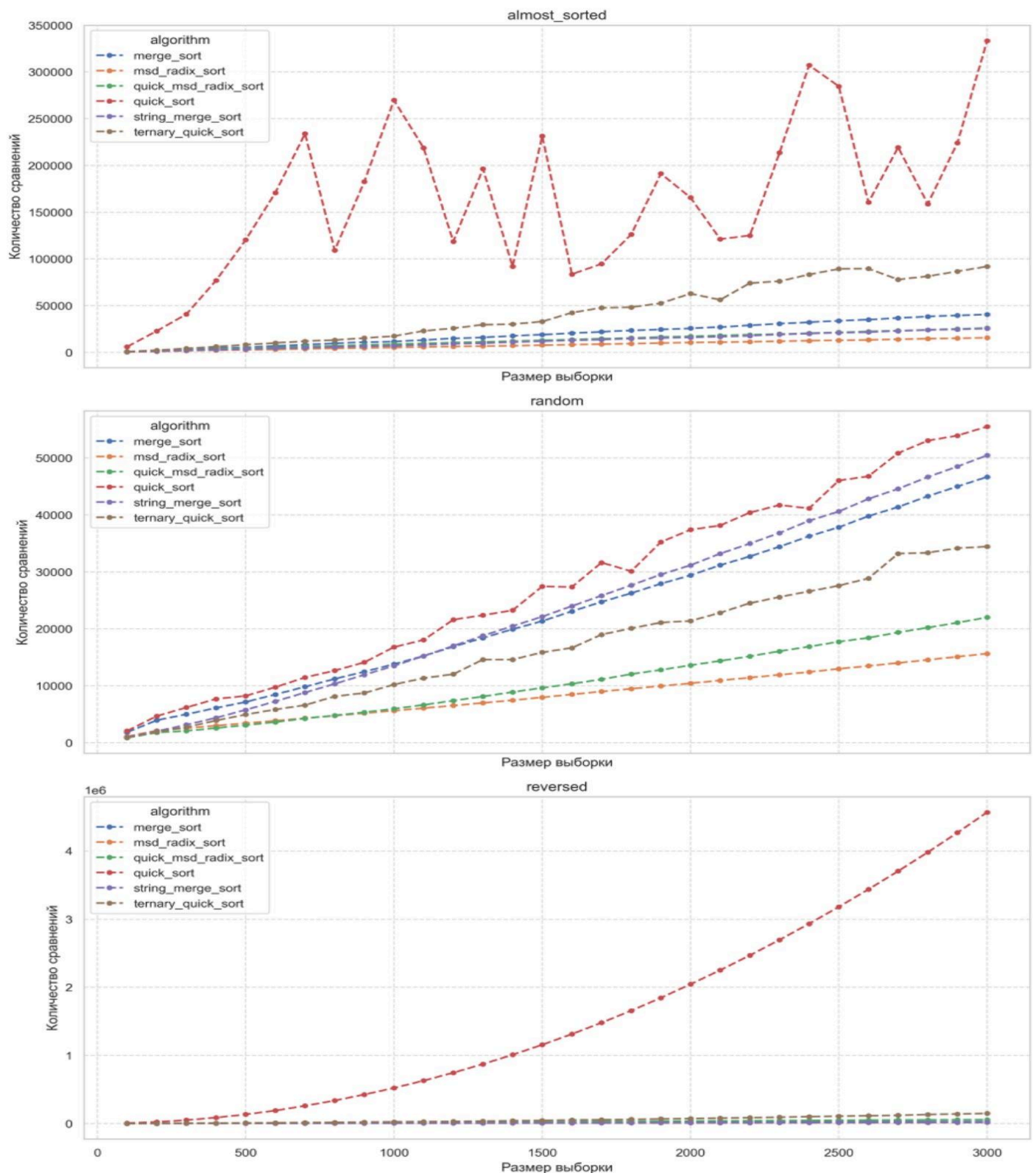
- **string_merge_sort.cpp** - STRING MERGESORT, использующий длину наибольшего общего префикса;
- **string_quick_sort.cpp** - тернарный STRING QUICKSORT;
- **msd_radix_sort.cpp** - MSD RADIX SORT без переключения на STRING QUICKSORT;
- **msd_radix_quick_sort.cpp** - MSD RADIX SORT с переключением на STRING QUICKSORT в случае, если длина сортируемого подмассива становится меньше мощности исходного алфавита.

Реализация класса StringSortTester находится в файле **StringSortTester.cpp**

Результаты замеров времени выполнения и количества символьных сравнений 4х алгоритмов в файле **result.csv**

Анализ количества посимвольных сравнений:

Анализ количества сравнений



Графики есть в репозитории в файле **comparisons.jpg**

1. По всем трем типам данных видно, что:

- Наибольшее число сравнений почти всегда дает стандартный QUICK SORT, особенно на обратно отсортированном массиве;
- MERGE SORT стабилен и лучше QUICK SORT, но хуже специализированных алгоритмов;
- Специализированные алгоритмы требуют значительно меньше символьных операций, разница сильнее при большем размере выборки.

2. Почти отсортированные данные

- QUICK SORT показывает очень большие значения числа сравнений: при увеличении размера выборки кривая растет до сотен тысяч сравнений, причем с сильными колебаниями. Это отражает чувствительность обычного quicksort к “неудачным” опорным элементам.
- MERGE SORT, STRING MERGE SORT и STRING QUICK SORT растут достаточно ровно и находятся на одном порядке величин.
- MSD RADIX SORT и особенно MSD RADIX+QUICK SORT - их кривые лежат ниже остальных и растут почти линейно.
- На почти отсортированных строках специализированные алгоритмы - лучше, чем QUICK SORT, и требуют существенно меньше символьных сравнений.

3. Случайные данные

- Все алгоритмы ведут себя без сильных скачков.
- QUICK SORT и MERGE SORT дают самые высокие кривые, коэффициент у них больше, чем у адаптированных методов.
- STRING MERGE SORT и STRING QUICK SORT находятся ниже стандартных: за счет учета общих префиксов они выполняют меньше символьных сравнений при тех же $O(n \log n)$.
- Лучшие по количеству операций - MSD RADIX SORT и MSD RADIX+QUICK SORT: их графики лежат внизу и растут почти линейно
- На случайных строках специализированные алгоритмы заметно выигрывают у стандартных; MSD RADIX+QUICK SORT - наиболее хороший для работы с символами.

4. Обратно отсортированные данные

- QUICK SORT показывает квадратичный рост числа сравнений: классический худший случай $O(n^2)$ при неудачном выборе опорного элемента.
- все остальные алгоритмы сохраняют почти линейно-логарифмическое или близкое к линейному поведение.
- Среди специализированных алгоритмов по числу символьных операций выигрывают MSD RADIX SORT и MSD RADIX+QUICK SORT

- На обратно отсортированном массиве преимущество специализированных алгоритмов по числу символьных сравнений становится максимальным - QUICK SORT реализует худший случай, а специализированные близки к теоретическим оценкам.

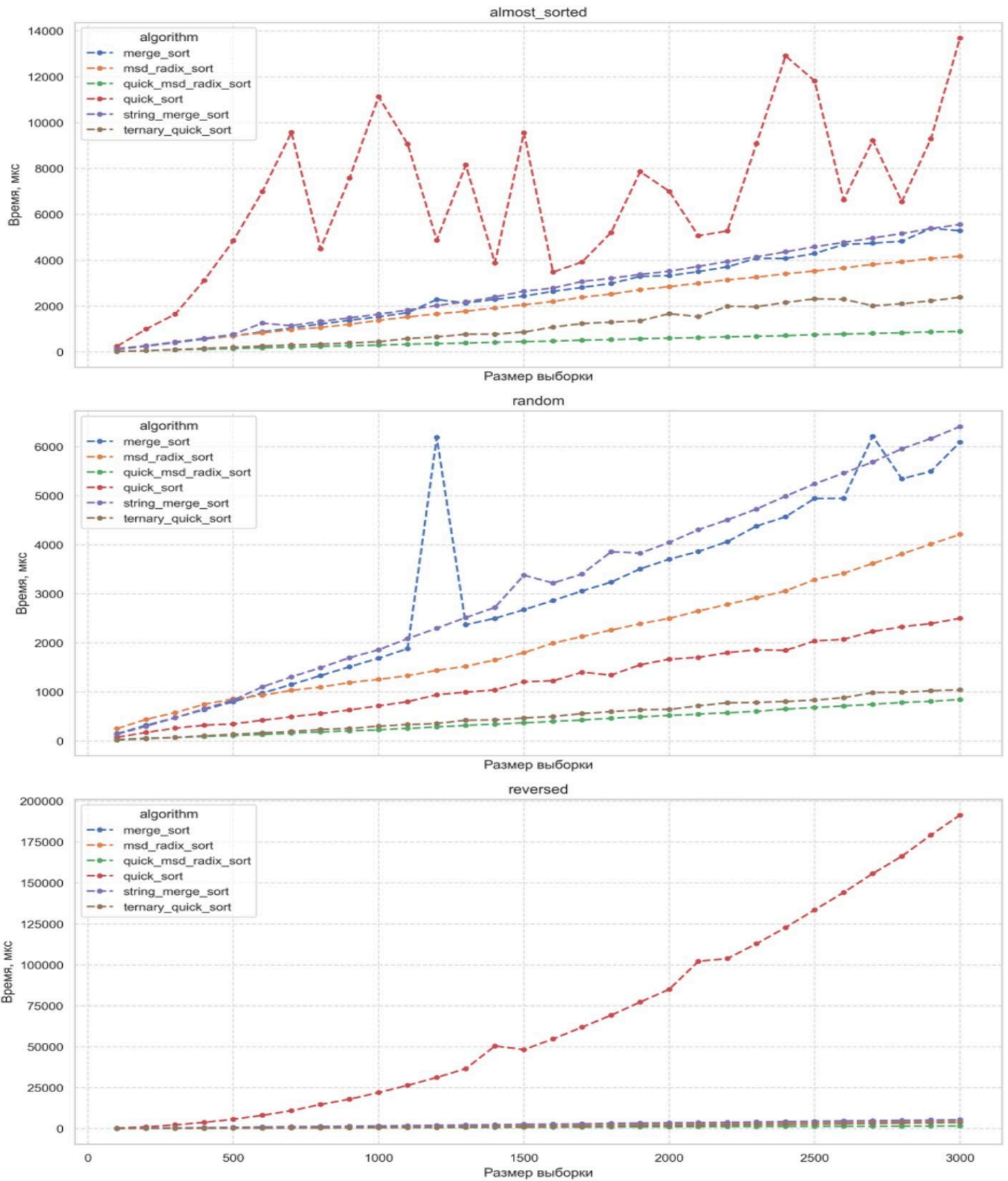
Вывод

1. QUICK SORT самый затратный по числу символьных сравнений.
2. MERGE SORT лучше QUICK SORT, но хуже специализированным алгоритмам.
3. STRING QUICK SORT и STRING MERGE SORT сокращают число символьных сравнений, что показывает большую эффективность - использовать общие префиксы строк.
4. MSD RADIX SORT и MSD RADIX+QUICK SORT дают наименьшее число символьных операций на всех типах данных; их графики растут почти линейно и лежат ниже остальных.

Анализ времени выполнения алгоритмов:

График находится в репозитории в файле **time.jpg**

Анализ времени выполнения



1. По всем трем графикам видно:

- Самый медленный алгоритм - QUICK SORT, особенно на обратно отсортированном массиве;
- MERGE SORT и STRING MERGE SORT дают близкие результаты;
- специализированные алгоритмы STRING QUICK SORT, MSD RADIX SORT и MSD RADIX+QUICK SORT демонстрируют наименьшее время выполнения.

2. Почти отсортированные данные

- QUICK SORT выше остальных и сильно колеблется: есть заметные минимумы и максимумы, показывающие нестабильность алгоритма на таких данных.
- MERGE SORT и STRING MERGE SORT растут медленнее специализированных алгоритмов, но быстрее QUICK SORT.
- STRING QUICK SORT быстрее стандартных алгоритмов, его кривая растет ровно.
- Самые низкое время у MSD RADIX SORT и MSD RADIX+QUICK SORT: специализированные алгоритмы дают минимальные значения времени.
- На почти отсортированных строках сильнее всего выигрывают radix алгоритмы, а QUICK SORT - наихудший.

3. Случайные данные

- MERGE SORT и STRING MERGE SORT дают близки друг с другом, есть резкие возрастания.
- QUICK SORT самый медленный: на больших размерах он выше всех.
- STRING QUICK SORT быстрее стандартных алгоритмов; рост времени более плавный.
- MSD RADIX SORT и MSD RADIX+QUICK SORT имеют лучшие результаты.
- На случайных строках специализированные алгоритмы выигрывают по времени относительно стандартных, при этом MSD RADIX+QUICK SORT наиболее эффективный.

4. Обратно отсортированные данные

- QUICK SORT - квадратичный рост, что соответствует теоретическому худшему случаю $O(n^2)$ для QUICK SORT.
- Все остальные алгоритмы имеют близкий к линейному рост.
- Среди них снова выигрывают по времени MSD RADIX SORT и MSD RADIX+QUICK SORT.
- На обратно отсортированных данных преимущество специализированных алгоритмов по времени становится максимальным, тогда как стандартный QUICK SORT реализует худший случай.

Вывод:

1. QUICK SORT дат наихудшее время на всех типах данных.
2. MERGE SORT и STRING MERGE SORT демонстрируют рост времени, близким к $O(n \log n)$.

3. STRING QUICK SORT по времени заметно выигрывает у стандартного QUICK SORT и часто лучше MERGE SORT, особенно на почти отсортированных и случайных строках.
4. MSD RADIX SORT и MSD RADIX+QUICK SORT дают минимальное время выполнения на всех типах входных данных; их кривые растут почти линейно и лежат ниже остальных алгоритмов, что соответствует теоретическим преимуществам поразрядной сортировки строк.